

技术附录

管线设计与复现说明

UTD 边缘融合初步实验

范围

本文记录上传实验代码的工程设计。内容包括数据扫描、特征提取、clean-condition baseline、degraded-sensing 测试、可选 Depth/RGB 扩展、输出文件，以及本地复现实验的步骤。

文档信息

管线名称	UTD Edge Fusion Preliminary Experiment
代码包	utd_edge_fusion_exp
核心模态	Skeleton, Inertial
可选模态	Depth, RGB/video
主要测试	Clean comparison; degraded-sensing robustness
Python 环境	Python 3.10-3.11; 使用 requirements.txt 安装依赖
日期	2026 年 4 月

目录

1	文档边界	1
1.1	目的	1
1.2	覆盖的代码包	1
2	管线概览	2
2.1	高层流程	2
2.2	实验问题	2
3	数据与配置设计	3
3.1	预期数据结构	3
3.2	核心配置	3
3.3	输出目录	3
4	模块设计	4
4.1	工具模块	4
4.2	可执行脚本	4
5	特征与融合设计	6
5.1	时序特征提取	6
5.2	退化模拟	6
5.3	可靠性感知表示	6
6	训练与评估设计	8
6.1	模型选择	8
6.2	指标	8
6.3	Clean-Condition Comparison	8
6.4	Degraded-Sensing Robustness	9
6.5	可选扩展模态	9

7 命令与复现	10
7.1 安装	10
7.2 设置数据根目录	10
7.3 核心管线	10
7.4 可选扩展管线	11
7.5 预期终端输出	11
8 生成文件	13
8.1 核心输出	13
8.2 扩展输出	13
8.3 Payload Estimate	14
9 注意事项与故障排查	15
9.1 运行前检查	15
9.2 常见失败情况	15
9.3 当前实现的限制	15
A 核心执行结构	17
A.1 核心 Runner	17
A.2 可选 Runner	17

1 文档边界

1.1 目的

本文从工程角度说明上传的实验代码。重点记录 pipeline 结构、各模块职责、所需输入、生成输出，以及本地复现步骤。

本文只覆盖当前研究原型：一个面向 UTD-style 多模态动作数据的实验管线，用于 clean-condition comparison 和 degraded-sensing comparison。

边界说明
当前代码是研究原型，适合做初步可行性测试和方法比较。它不是生产部署包，也不是完整的实时 edge system 或完整 benchmark suite。

1.2 覆盖的代码包

本文覆盖以下代码包：

```
1 utd_edge_fusion_exp/  
2   config.py  
3   utils_io.py  
4   utils_features.py  
5   utils_models.py  
6   00_check_env.py  
7   01_scan_dataset.py  
8   02_extract_features.py  
9   03_train_clean_baselines.py  
10  04_eval_degraded_sensing.py  
11  05_export_tables_figures.py  
12  06_optional_extract_depth_rgb_features.py  
13  07_train_extended_modalities.py  
14  08_eval_extended_degraded_robustness.py  
15  run_all_core.py  
16  run_extended.py  
17  requirements.txt  
18  README.md
```

2 管线概览

2.1 高层流程

管线采用分阶段结构。每个脚本把中间结果写入 `outputs/`，因此后续步骤可以单独重跑，不需要每次从头执行。

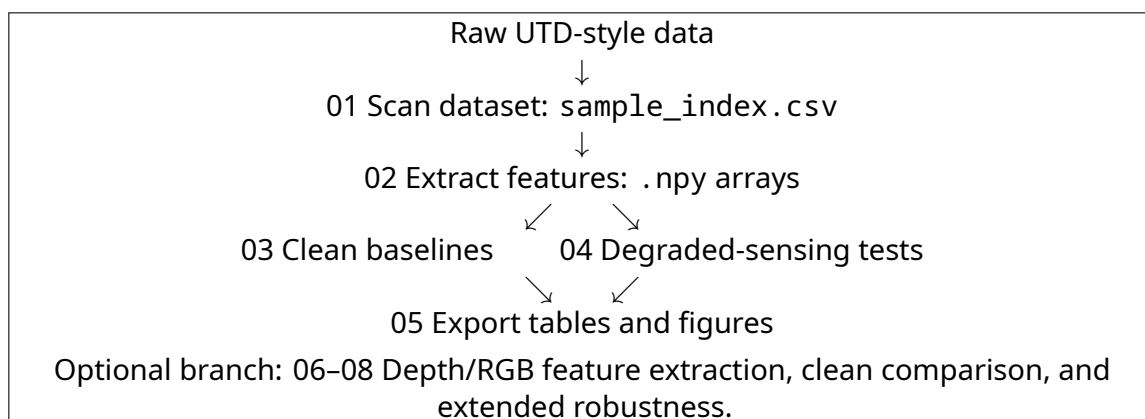


图 2.1: 上传脚本实现的 pipeline flow。

2.2 实验问题

这套代码检查一个比较聚焦的问题：从多个 sensor stream 中提取 lightweight feature 后，single-modality model、simple early fusion 和 reliability-aware fusion 在 clean 与 degraded sensing 条件下表现如何。

测试内容包括：

- Skeleton-only baseline;
- Inertial-only baseline;
- 通过 feature vector concatenation 实现的 early fusion;
- reliability-aware fusion，即使用模拟 reliability indicator 对 feature vector 加权;
- 如果 Depth 与 RGB/video 文件可用，则运行 optional extended experiments.

3 数据与配置设计

3.1 预期数据结构

默认数据根目录在 `config.py` 中设置，也可以通过环境变量 `UTD_DATA_ROOT` 覆盖。核心实验预期存在 `Skeleton/` 和 `Inertial/` 两类文件夹。Depth 与 RGB/video 为可选扩展。

```
1 DATASET/  
2   Skeleton/  
3   Inertial/  
4   Depth/      # optional  
5   RGB/        # optional
```

文件名需要包含 action、subject 和 trial 信息。`01_scan_dataset.py` 会读取文件名并建立统一的 sample index。后续步骤依赖这个 index 来对齐不同模态。

3.2 核心配置

主要配置集中在 `config.py` 中。下面只列与复现最相关的项目。

```
1 DATA_ROOT = Path(os.environ.get("UTD_DATA_ROOT", r"/path/to/UTD_dataset/DATASET"))  
2 OUTPUT_DIR = Path("outputs")  
3 RANDOM_SEEDS = [13, 42, 2026]  
4 TEST_SUBJECTS = [1, 3, 5, 7]  
5 ENABLE_DEPTH_FEATURES = False  
6 ENABLE_RGB_FEATURES = False
```

代码 3.1: `config.py` 中的核心配置

3.3 输出目录

管线把输出分成 index、feature、result、figure 等几类目录。

目录	用途
<code>outputs/index/</code>	存放 <code>sample_index.csv</code> 和数据扫描结果。
<code>outputs/features/</code>	存放 <code>Skeleton/Inertial/Depth/RGB</code> 的 feature arrays。
<code>outputs/results/</code>	存放 clean 与 degraded condition 下的 result tables。
<code>outputs/figures/</code>	存放 robustness curves、comparison figures 与导出的 PDF/PNG。

4 模块设计

4.1 工具模块

模块	层级	主要职责
utils_io.py	I/O layer	读取 .mat 文件、建立目录、保存 CSV/NumPy 文件。
utils_features.py	Feature layer	从 Skeleton、Inertial 和 optional modalities 提取 compact statistics。
utils_models.py	Model layer	构建 scikit-learn model、运行训练、汇总 metrics。
config.py	Config layer	保存路径、随机种子、subject split、可选模态开关和输出目录。

4.2 可执行脚本

脚本	主要输入	主要输出 / 功能
00_check_env.py	本地 Python 环境	打印 Python 版本、data path、output path 和 package availability。
01_scan_dataset.py	Raw dataset folders	写入 outputs/index/sample_index.csv。
02_extract_features.py	sample_index.csv	写入 Skeleton/Inertial feature arrays, 以及 label/subject arrays。
03_train_clean_baselines.py	Feature arrays	训练 clean-condition baselines, 并写出 clean result tables。
04_eval_degraded_sensing.py	Feature arrays	模拟 sensing degradation, 并写出 robustness result tables。
05_export_tables_figures.py	Result CSV files	导出 LaTeX tables、robustness curves 和 payload estimate。
06_optional_extract_depth_rgb_features.py	Optional paths	当可选文件存在时, 提取 Depth/RGB features。
07_train_extended_modalities.py	Core and optional features	对可选模态参与时的 clean-condition setting 做比较。

08_eval_extended_degraded_robustness.py	Extended features	对 extended modality setting 做 degraded robustness test。
---	-------------------	---

5 特征与融合设计

5.1 时序特征提取

当前实现使用 compact handcrafted statistics，而不是完整 sequence model。这样做的目的不是追求最终模型性能，而是让不同模态和 degradation setting 可以快速比较。

```
1 def compact_stats(x):
2     return np.concatenate([
3         np.mean(x, axis=0),
4         np.std(x, axis=0),
5         np.min(x, axis=0),
6         np.max(x, axis=0),
7     ])
```

代码 5.1: utils_features.py 中使用的特征统计

Skeleton features 来自 joint coordinate sequence。Inertial features 来自 accelerometer/gyro-scope sequence。Depth/RGB features 是 optional lightweight summary；如果对应文件缺失，core pipeline 仍然可以运行。

5.2 退化模拟

Degraded-sensing test 在 feature level 模拟 sensing quality 下降。它适合做 controlled comparison，但不等同于完整 sensor-level corruption modelling。

```
1 def degrade_features(x, mode, strength, rng):
2     if mode == "noise":
3         return x + rng.normal(0, strength, size=x.shape)
4     if mode == "dropout":
5         mask = rng.random(size=x.shape) > strength
6         return x * mask
7     if mode == "scale":
8         return x * (1.0 - strength)
9     return x
```

代码 5.2: Feature-level degradation simulation

5.3 可靠性感知表示

Reliability-aware fusion 在 concatenated feature 前加入简单的 modality reliability weighting。它的作用是观察当某个 sensor stream 质量下降时，fusion representation 是否比 plain concate-

nation 更稳定。

```
1 def reliability_weighted_concat(features, reliabilities):
2     weighted = []
3     for name, x in features.items():
4         r = reliabilities.get(name, 1.0)
5         weighted.append(x * r)
6     return np.concatenate(weighted, axis=1)
```

代码 5.3: Core reliability-aware fusion representation

当前 reliability value 是实验设置中的简化指标。后续如果扩展为正式研究，可以把它替换为 sensor quality estimator、missing-modality indicator 或 calibration-based confidence。

6 训练与评估设计

6.1 模型选择

当前代码使用 scikit-learn 模型进行快速 baseline comparison。主要原因是数据规模较小、feature dimension 可控，并且需要重复运行多组 degradation setting。

```
1 def make_model(kind="logreg"):
2     if kind == "logreg":
3         return LogisticRegression(max_iter=2000, class_weight="balanced")
4     if kind == "svm":
5         return LinearSVC(class_weight="balanced")
6     if kind == "rf":
7         return RandomForestClassifier(n_estimators=300, random_state=42)
8     raise ValueError(kind)
```

代码 6.1: utils_models.py 中的模型选择逻辑

模型层故意保持轻量，便于观察 pipeline、feature、modality 和 degradation 设置的影响。更重的 sequence model 可以作为后续扩展，而不是当前附件的重点。

6.2 指标

代码主要报告 accuracy 和 macro-F1。Accuracy 用于快速检查整体分类情况，macro-F1 用于减少类别不平衡带来的偏差。

如果这套实验结果将被用作正式研究证据，建议再补充 per-class precision/recall、confusion matrix、cross-subject folds 和 confidence interval。

6.3 Clean-Condition Comparison

03_train_clean_baselines.py 比较 clean input 下的几种设置：

- Skeleton-only;
- Inertial-only;
- Skeleton + Inertial early fusion;
- Skeleton + Inertial reliability-aware fusion。

每个 setting 在多个 random seed 下运行，并把结果写入 outputs/results/。

6.4 Degraded-Sensing Robustness

04_eval_degraded_sensing.py 使用相同 feature arrays，模拟 noise、dropout 或 scale degradation。输出用于比较 clean 与 degraded setting 之间的 performance drop。

6.5 可选扩展模态

Depth/RGB 扩展由 06-08 三个脚本处理。它们不是 core pipeline 的必要部分。只有当相应数据文件、路径和依赖准备好时，才建议开启。

7 命令与复现

7.1 安装

基础依赖通过 requirements.txt 安装。

```
1 python -m venv .venv
2 .venv\Scripts\activate
3 pip install -r requirements.txt
```

代码 7.1: Basic installation

macOS 或 Linux 下可以使用：

```
1 python3 -m venv .venv
2 source .venv/bin/activate
3 pip install -r requirements.txt
```

如果要使用 RGB/video optional branch，需要额外安装 OpenCV。

```
1 pip install opencv-python
```

代码 7.2: Optional RGB/video dependency

7.2 设置数据根目录

Windows Command Prompt:

```
1 set UTD_DATA_ROOT=C:\path\to\UTD_dataset\DATASET
```

PowerShell:

```
1 $env:UTD_DATA_ROOT="C:\path\to\UTD_dataset\DATASET"
```

macOS 或 Linux:

```
1 export UTD_DATA_ROOT=/path/to/UTD_dataset/DATASET
```

如果没有设置环境变量，代码会使用 config.py 中的默认路径。

7.3 核心管线

最直接的运行方式是执行 core runner。

```
1 python run_all_core.py
```

等价的手动步骤如下：

```
1 python 00_check_env.py
2 python 01_scan_dataset.py
3 python 02_extract_features.py
4 python 03_train_clean_baselines.py
5 python 04_eval_degraded_sensing.py
6 python 05_export_tables_figures.py
```

7.4 可选扩展管线

Depth/RGB 扩展管线在 core features 已经成功生成之后运行。

```
1 python run_extended.py
```

也可以手动运行：

```
1 python 06_optional_extract_depth_rgb_features.py
2 python 07_train_extended_modalities.py
3 python 08_eval_extended_degraded_robustness.py
```

7.5 预期终端输出

下列输出格式用于快速确认脚本正在按预期运行。具体路径、样本数和数值会随本地数据而变化。

```
1 [ENV] Python: 3.10.x
2 [ENV] DATA_ROOT: /path/to/UTD_dataset/DATASET
3 [ENV] OUTPUT_DIR: outputs
4 [OK] numpy available
5 [OK] pandas available
6 [OK] sklearn available
```

代码 7.3: Environment check message pattern

```
1 [SCAN] Skeleton folder found
2 [SCAN] Inertial folder found
3 [SCAN] matched samples: 0000
4 [WRITE] outputs/index/sample_index.csv
```

代码 7.4: Dataset scan message pattern

```
1 [LOAD] outputs/index/sample_index.csv
2 [FEATURE] extracting Skeleton features
3 [FEATURE] extracting Inertial features
4 [WRITE] outputs/features/X_skeleton.npy
5 [WRITE] outputs/features/X_inertial.npy
```

代码 7.5: Feature extraction message pattern

```
1 [TRAIN] setting=skeleton_only seed=13
2 [TRAIN] setting=inertial_only seed=13
3 [TRAIN] setting=early_fusion seed=13
4 [TRAIN] setting=reliability_fusion seed=13
5 [WRITE] outputs/results/clean_results_by_seed.csv
```

代码 7.6: Clean-condition result message pattern

```
1 [OPTIONAL] Depth features enabled: False
2 [OPTIONAL] RGB features enabled: False
3 [SKIP] optional modality not available
```

代码 7.7: Extended modality message pattern

8 生成文件

8.1 核心输出

Core pipeline 完成后，主要输出包括：

```
1 outputs/  
2   index/  
3     sample_index.csv  
4   features/  
5     X_skeleton.npy  
6     X_inertial.npy  
7     y.npy  
8     subjects.npy  
9   results/  
10    clean_results_by_seed.csv  
11    clean_results_summary.csv  
12    degraded_results_by_seed.csv  
13    degraded_results_summary.csv  
14    table_clean_latex.txt  
15    table_degraded_latex.txt  
16   figures/  
17     robustness_accuracy.pdf  
18     robustness_accuracy.png  
19     robustness_macro_f1.pdf  
20     robustness_macro_f1.png  
21     edge_payload_estimate.txt
```

8.2 扩展输出

如果 optional branch 可运行，可能会额外生成：

```
1 outputs/  
2   features/  
3     X_depth.npy  
4     X_rgb.npy  
5   results/  
6     extended_clean_results_by_seed.csv  
7     extended_clean_results_summary.csv  
8     extended_degraded_results_by_seed.csv  
9     extended_degraded_results_summary.csv  
10    table_extended_degraded_latex.txt  
11   figures/  
12     extended_robustness_core_splusi.pdf
```

```
13 extended_robustness_core_splusi.png
14 extended_robustness_all_available.pdf
15 extended_robustness_all_available.png
16 extended_reliability_comparison.pdf
17 extended_reliability_comparison.png
```

8.3 Payload Estimate

`05_export_tables_figures.py` 会估算 raw feature upload 与 edge event summary upload 之间的通信 payload 差异。这个估算不是网络 benchmark，只是一个设计说明：在 edge 侧输出 event-level summary 可能减少传输数据量。

9 注意事项与故障排查

9.1 运行前检查

- 确认 DATA_ROOT 指向正确的数据集目录。
- 确认文件名模式中包含 action、subject 和 trial 编号。
- 运行完整管线前，先执行 00_check_env.py。
- 在 optional 文件和依赖准备好之前，不要同时开启 Depth/RGB 相关开关。
- 检查当前 subject split 是否符合本次实验目的。

9.2 常见失败情况

现象	可能原因	处理方式
DATA_ROOT does not exist	默认路径错误，或没有设置环境变量	设置 UTD_DATA_ROOT，或修改 config.py。
Missing Skeleton/Inertial samples	文件夹名或文件名模式不匹配	检查 Skeleton/、Inertial/ 和文件名拼写。
Cannot find d_skel 或 d_iner	MATLAB 变量名与代码预期 key 不同	检查 .mat 文件，并调整 load_mat_var 的使用方式。
Optional RGB skipped	OpenCV 缺失，或 video path 无法读取	安装 opencv-python，并检查视频格式。
Optional modality skipped	Optional feature IDs 无法与 master sample IDs 对齐	检查 optional files 是否使用相同的 action/subject/trial 标识。
分数异常偏低	特征较小、subject split 较难、数据不匹配或 degradation 过强	检查 per-class results，并使用多个 random seed 重跑。

9.3 当前实现的限制

- 特征提取是 handcrafted 且 compact 的；它没有像 LSTM、Transformer 或 graph model 那样建模完整时序动态。

- Degradation 在 feature level 模拟。它适合 controlled comparison, 但不能替代 sensor-level corruption modelling。
- 当前评估主要报告 accuracy 和 macro-F1。完整研究可能还需要更多诊断指标。
- Subject split 固定在 config.py 中。如果结果要作为正式证据, 还需要更多 split design。
- Optional Depth/RGB features 是 lightweight summaries, 应被视为 extension check, 而不是最终 video-understanding model。

A 核心执行结构

A.1 核心 Runner

Core runner 会按顺序执行核心脚本。如果某一步失败，runner 会停止。

```
1 steps = [  
2     "00_check_env.py",  
3     "01_scan_dataset.py",  
4     "02_extract_features.py",  
5     "03_train_clean_baselines.py",  
6     "04_eval_degraded_sensing.py",  
7     "05_export_tables_figures.py",  
8 ]  
9  
10 for step in steps:  
11     print("\n" + "=" * 80)  
12     print("Running", step)  
13     print("=" * 80)  
14     result = subprocess.run([sys.executable, step])  
15     if result.returncode != 0:  
16         print(f"[STOP] {step} failed with code {result.returncode}")  
17         raise SystemExit(result.returncode)
```

代码 A.1: run_all_core.py

A.2 可选 Runner

Optional runner 默认 core feature extraction 已经成功完成。

```
1 for script in [  
2     "06_optional_extract_depth_rgb_features.py",  
3     "07_train_extended_modalities.py",  
4     "08_eval_extended_degraded_robustness.py",  
5 ]:  
6     print("\n" + "=" * 80)  
7     print(f"Running {script}")  
8     print("=" * 80)  
9     subprocess.run([sys.executable, script], check=True)
```

代码 A.2: run_extended.py